HamdhyTech Student Attendance System

Codex-Ready Project Specification and Application Creation Guide

Purpose of this document

Upload this PDF to Codex together with the project repository. It defines the required applications, users, screens, workflows, database model, security rules, notification logic, cost controls, development phases, and acceptance tests. Codex should treat it as the primary product specification.

Document item	Value
Developer / owner	HamdhyTech
Solution	Student attendance management and parent notification system
Applications	Main App and Parent App
Primary platform	Android-first Flutter mobile applications
Backend	Firebase Authentication, Cloud Firestore, Firebase Cloud Messaging
Student photos	Not included
Default alerts	Free push notifications; SMS optional and paid
Version	1.0 - Final planning specification
Prepared date	1 August 2026

Confidential project planning document

1. Instructions for Codex

Primary instruction

Read this entire specification before generating code. Do not attempt to build every feature in one uncontrolled change. Create the system in the development phases listed in this document, run analysis and tests after every phase, and report assumptions or deviations clearly.

- **Required technology:** Flutter and Dart for both mobile apps, using one shared Firebase project.
- **Repository design:** A monorepo containing main_app, parent_app, shared Dart package, Firebase rules, indexes, backend notification code, tests, and documentation.
- **Cost objective:** Keep the core system within free service quotas. SMS is optional and is the only intended recurring paid feature.
- **Privacy rule:** Do not add student photographs or cloud image storage.
- **Security rule:** Never hardcode administrator credentials, API secrets, service-account files, or plain-text passwords.
- **Scanner rule:** The QR scanner opens once and continuously scans students without requiring a confirmation tap after every valid scan.
- **Data isolation:** Every institute must be strictly isolated from all other institutes through backend security rules, not only through the UI.
- **Quality rule:** A feature is not complete until UI, validation, loading states, errors, security, tests, and documentation are included.

2. Product Vision

The HamdhyTech Student Attendance System enables educational institutes to record student arrival and departure using unique QR codes. Teachers use the Main App to scan attendance. Parents use the Parent App to see live attendance information and receive push notifications similar to WhatsApp notifications. Optional SMS can be enabled only for institutes that purchase an SMS package.

2.1 Core outcomes

- Teachers can record attendance rapidly with continuous QR scanning.
- Institute administrators can manage teachers, classes, students, attendance, and reports.
- HamdhyTech can create and control institutes from a Super Admin account.
- Parents receive clear arrival and departure information with minimal navigation.
- Parent mobile numbers are collected for account linking, contact, and optional SMS.
- The solution avoids student photo storage and unnecessary paid services.
- Attendance remains saved even if SMS or push notification delivery fails.

3. Solution Architecture

```
HamdhyTech Attendance System
|
+-- Main App
|   +-- Super Admin
|   +-- Institute Admin
|   +-- Teacher
```

```
|
+-- Parent App
|   +-- Parent / Guardian
|
+-- Shared Firebase Project
    +-- Firebase Authentication
    +-- Cloud Firestore
    +-- Firebase Cloud Messaging
    +-- Secure notification backend
    +-- Firestore Security Rules and indexes
```

Both apps use the same database but expose different screens and permissions. A user role and institute membership must be verified after authentication before data is displayed.

4. Application and Package Decisions

Item	Recommended value
Main App name	HamdhyTech Attendance
Parent App name	HamdhyTech Attendance Parent
Main App package	com.hamdhytech.attendance.main
Parent App package	com.hamdhytech.attendance.parent
Shared package	hamdhytech_attendance_core
Default language	English
Future languages	Sinhala and Tamil
UI framework	Material 3
Orientation	Portrait-first; scanner may support portrait and landscape
Minimum release target	Android APK; Play Store publication optional

5. User Roles and Permission Model

Role	Scope	Main permissions
Super Admin	All institutes	Create/suspend institutes, create institute admins, view system-wide summaries, control SMS/push permissions, reset accounts, inspect audit and delivery logs.
Institute Admin	One institute only	Manage teachers, classes and students; assign teachers; view all institute attendance and reports; approve parent links; reset teacher accounts.
Teacher	Assigned institute/classes	View assigned classes, add students if permitted, generate QR cards, run attendance sessions, correct records with a reason, view/export class reports.
Parent/Guardian	Verified linked students only	View linked children, classes, arrival/departure times, history and notifications. Cannot edit attendance.

5.1 Account creation hierarchy

```
HamdhyTech Super Admin
  -> creates Institute
```

-> creates initial Institute Admin account

Institute Admin

-> creates Teacher accounts

Teacher / Institute Admin

-> creates Students and Classes

Parent

-> registers separately and securely links a Student

- Temporary passwords must be changed at first login.
- Passwords must be managed through Firebase Authentication and never stored as readable fields in Firestore.
- The Super Admin account must be created securely outside the released application or through a protected provisioning process.
- Role checks must be enforced in Firestore Security Rules and backend functions.

6. MAIN APP - Complete Functional Specification

The Main App contains three dashboards. The correct dashboard opens after authentication and role verification.

6.1 Shared Main App login

- Username/email and password login.
- Forgot-password flow.
- First-login forced password change for generated accounts.
- Account active/suspended status check.
- Role and institute membership retrieval.
- Friendly errors for wrong credentials, inactive institute, disabled user, and unavailable network.
- No visible or hidden hardcoded administrator password.

6.2 Super Admin dashboard

- Dashboard totals: active institutes, suspended institutes, teachers, students, classes, attendance records today, push notifications and SMS usage.
- Institute search, filtering and sorting.
- Create, edit, activate, suspend and archive an institute.
- Create the first Institute Admin account with a temporary password.
- Open any institute to view its administrators, teachers, classes, students, attendance summaries and system activity.
- Enable or disable Parent App access, push notifications and SMS independently for each institute.
- Set SMS monthly allowance, current package name, overage policy and warning threshold.
- Allow or prevent paid extra SMS after the package allowance is consumed.
- Reset Institute Admin access without revealing existing passwords.
- View delivery failures and audit logs.

6.3 Create institute form

Field	Rule
Institute name	Required
Institute code	Unique; auto-generated with editable option
Address	Optional

Contact mobile	Required
Contact email	Optional
Institute Admin name	Required
Institute Admin username/email	Required and unique
Temporary password	Generated securely or entered with strength rules
SMS enabled	Default OFF
Push enabled	Default ON
Parent App enabled	Default ON
Account status	Active or suspended
Expiry/package data	Optional for future commercial plans

6.4 Institute Admin dashboard

- Today's attendance summary and upcoming classes.
- Create, edit, disable and restore teachers.
- Create and manage classes, schedules and teacher assignments.
- Create, edit, archive and transfer students between classes.
- View all institute attendance sessions and records.
- Approve or reject parent-student linking requests.
- View push/SMS availability but cannot override a Super Admin restriction.
- Export reports to CSV and PDF.
- View audit logs for important changes.

6.5 Teacher account management

Field / permission	Requirement
Teacher full name	Required
Username/email	Required and unique
Mobile number	Optional
Temporary password	Required; force change at first login
Assigned classes	One or more
Can create classes	Configurable
Can add/edit students	Configurable
Can correct attendance	Configurable
Can export reports	Configurable
Status	Active or disabled

6.6 Class management

- Create class name, subject, grade/level, class code, teacher(s), days, start time, expected end time, room/location and academic year.
- Class code must be unique within the institute.
- Assign existing students or create new students.
- Archive a class without deleting its historical attendance.
- Allow multiple teachers only when explicitly assigned.

6.7 Student management

Student field	Requirement
Student number	Required; entered by institute or generated if blank; unique within institute
Full name	Required

Primary parent/guardian name	Required
Primary parent mobile number	Required
Secondary parent name/mobile	Optional
Preferred message language	English initially; extensible
Registered classes	One or more
Push alerts enabled	Default ON after parent links
SMS enabled	Default OFF; requires institute permission and consent
Status	Active, inactive or archived
Photo	Not collected and not stored

Mandatory mobile number

The primary parent mobile number must be collected for every student. Validate Sri Lankan and international formats without permanently forcing one country code. Store a normalized version for matching and an original display version.

6.8 QR code generation and QR card

- Generate one secure QR token per student.
- Do not encode the student name, parent phone number, address or other private information directly in the QR.
- QR payload should contain a versioned opaque token or signed identifier.
- Allow display, regenerate, disable, share and export as PDF.
- When regenerated, the old token becomes invalid.
- QR card shows institute name, student name, student number, optional class list and the QR code. No student photo.

  Example logical QR payload (not private data):

  HTA1:<opaque-student-token>

6.9 Continuous QR attendance scanner

Confirmed scanner behaviour

The teacher selects a class and Entry or Departure once. The camera remains open and automatically records every valid QR. No confirmation button is required for each student. The latest scanned student details appear below the camera while the scanner immediately becomes ready for the next student.

1. Teacher opens the assigned class.
2. Teacher taps Mark Attendance.
3. Teacher selects Entry or Departure mode.
4. Teacher starts the scanner once.
5. The app detects a QR automatically.
6. The app validates institute, class, student, token and attendance state.
7. The valid record is saved automatically.
8. The screen shows a green success state, optional sound/vibration, student name, student number, class, mode and time below the camera.
9. The scanner remains open and unlocks for the next student after a short cooldown.

10. The teacher ends the session and views a summary.

6.10 Scanner validation rules

- QR token must exist and be active.
- Student must belong to the selected institute.
- Student must be registered in the selected class.
- Student account must be active.
- Duplicate entry must be blocked.
- Departure before entry must be blocked.
- Second departure must be blocked.
- A short per-token cooldown prevents one QR being scanned repeatedly.
- Invalid, expired or other-institute QR codes show a red message but do not close the scanner.
- Network errors must not silently create duplicate records.
- Optional offline queue can be added after the online version is stable.

6.11 Attendance record behaviour

- Attendance session is created for a class and date.
- Entry and departure are stored in one logical student/session record where possible to reduce database writes.
- Use server timestamps for authoritative time; device time may be displayed only as temporary feedback.
- Record who marked or corrected attendance.
- Manual corrections require a reason and create an audit entry.
- Attendance saving is independent from notification sending.
- If notifications fail, attendance remains successfully recorded and delivery can be retried.

6.12 Attendance messages

All notifications and SMS messages must be attractive, clear and institute-branded. The institute name should appear first.

Event	Recommended message
Arrival push	[Institute Name] Dear Parent, [Student Name] has arrived for [Class Name] at [Time].
Departure push	[Institute Name] Dear Parent, [Student Name] departed from [Class Name] at [Time].
Late arrival	[Institute Name] [Student Name] arrived late for [Class Name] at [Time].
Short SMS arrival	[Institute Name]: [Student Name] arrived for [Class Name] at [Time].
Short SMS departure	[Institute Name]: [Student Name] departed from [Class Name] at [Time].

- Keep SMS templates short because long or Unicode messages may consume multiple SMS units.
- Allow institute-level template customization within safe placeholder limits.

- Store notification and SMS delivery logs without storing secret provider credentials in the mobile apps.

6.13 SMS controls

Send SMS only when ALL are true:

1. Super Admin enabled SMS for the institute.
2. Institute account is active.
3. Student SMS preference is enabled and consent recorded.
4. A valid parent mobile number exists.
5. SMS allowance remains OR paid overage is allowed.
6. The attendance event type is enabled.

- When SMS is OFF, attendance and push notifications continue normally.
- Teachers see a clear status such as “SMS unavailable for this institute”.
- Super Admin sees monthly allowance, used, remaining, failed and estimated extra SMS.
- Optional Notify.lk integration must be implemented through a secure backend, not directly from the APK.

6.14 Reports and exports

- Daily class attendance report.
- Student attendance history.
- Arrival/departure report.
- Absent and late lists.
- Monthly attendance percentage.
- CSV export suitable for Google Sheets.
- PDF report generation on-device where practical.
- Google Sheets direct synchronization is a later optional feature; Firestore remains the source of truth.

7. PARENT APP - Complete Functional Specification

The Parent App must be simpler than the Main App. A parent should see the child’s current status and latest attendance information immediately after opening the app.

7.1 Parent registration and login

- Email/password registration and login for the low-cost version.
- Parent full name and mobile number are required.
- Mobile number should be normalized and compared with the student’s registered parent number during linking.
- Phone OTP is not required in version one because it may introduce SMS authentication cost.
- Forgot-password support.
- Notification permission request with a clear explanation.

7.2 Secure student linking

11. Parent enters the institute code.
12. Parent enters the student number.
13. The system verifies that the parent account mobile matches a registered parent mobile, or requests an institute approval/linking PIN.

14. A parent-student link request is created.
15. The institute approves automatically only when strong matching rules pass; otherwise the Institute Admin approves manually.
16. After approval, the parent can access only that student's permitted information.

- Do not grant access using institute code and student number alone.
- Allow one parent account to link multiple children.
- Allow a student to have approved primary and secondary guardians.
- Allow the institute to revoke a link.

7.3 Parent dashboard

- Large student selector when multiple children are linked.
- Student name and student number; no photograph.
- Current status: Not arrived, In class, Departed, No class today, or Unknown.
- Today's classes with scheduled times.
- Latest arrival and departure time.
- Latest notification card.
- Attendance percentage and simple monthly overview.
- One-tap attendance history and institute contact.

7.4 Push notification behaviour

- Use Firebase Cloud Messaging for WhatsApp-style push notifications.
- Notifications should work when the app is backgrounded or closed, subject to device permissions and internet connectivity.
- Tapping the notification opens the exact child and class attendance record.
- Store an in-app notification history so information remains available after the notification is dismissed.
- Handle token refresh and multiple devices per parent.
- Allow parents to control arrival and departure push preferences where institute policy permits.

7.5 Parent App screens

Screen	Purpose
Splash / session check	Restore a valid session and open the dashboard.
Login / registration	Create or access a parent account.
Link student	Securely request access using institute and student details.
Dashboard	Show immediate current status and today's classes.
Student overview	Show classes, current state and attendance summary.
Attendance history	Filter by class and date.
Notifications	View arrival/departure alerts.
Linked children	Add, switch or remove child links.
Institute contact	Display institute-provided contact details.
Settings	Notification preferences, password and profile.

8. User Experience Requirements

- Use Material 3 and a consistent HamdhyTech theme.
- Parent App must use large touch targets, readable text and minimal steps.
- Do not crowd dashboards with technical information.

- Use green for successful arrival/confirmed states, orange for late/warnings and red for errors/absence.
- Do not depend on color alone; include icons and text labels.
- Show loading skeletons or progress indicators for network operations.
- Provide useful empty states such as “No classes today” and “No attendance records yet”.
- Confirmation dialogs are required only for destructive or irreversible actions.
- The scanner must never require repeated tapping between students.
- Support common small Android screens and accessibility text scaling.

9. Firestore Data Model

users/{uid}
institutes/{instituteId}
instituteMembers/{membershipId}
classes/{classId}
students/{studentId}
classStudents/{classStudentId}
attendanceSessions/{sessionId}
attendanceRecords/{recordId}
parentStudentLinks/{linkId}
notificationTokens/{tokenId}
notifications/{notificationId}
smsLogs/{smsLogId}
auditLogs/{auditLogId}
appSettings/global

9.1 Important field examples

Collection	Important fields
users	uid, displayName, email, mobileNormalized, role, active, mustChangePassword, createdAt
institutes	name, code, status, smsEnabled, pushEnabled, parentAppEnabled, smsLimit, smsUsed, allowSmsOverage
instituteMembers	uid, instituteId, role, permissions, active
classes	instituteId, code, name, subject, grade, teacherIds, schedule, active
students	instituteId, studentNumber, fullName, parent names/mobiles, qrTokenHash, smsEnabled, active
attendanceSessions	instituteId, classId, dateKey, mode, startedBy, startedAt, endedAt
attendanceRecords	instituteId, classId, sessionId, studentId, entryAt, departureAt, status, markedBy, correction data
parentStudentLinks	parentUid, studentId, instituteId, relationship, status, approvedBy, approvedAt
notifications	parentUid, studentId, eventType, title, body, attendanceRecordId, sentAt, readAt
smsLogs	instituteId, studentId, mobileMasked, eventType, providerId, units, status, error, createdAt
auditLogs	actorUid, instituteId, action, entityType, entityId, before/after summary, timestamp

9.2 Data efficiency rules

- Use stable IDs and indexed query fields.
- Avoid loading every student or record when only one class/date is needed.
- Paginate long lists and reports.
- Use one attendance record per student/class/date where practical, updating departure in the same document.
- Do not create unnecessary real-time listeners on large collections.
- Detach listeners when screens close.
- Cache safe reference data locally.
- Use aggregate counters carefully and transactionally where required.

10. Security and Privacy Requirements

- All access requires Firebase Authentication except public app metadata that is intentionally exposed.
- Security Rules verify role, institute membership and ownership for every read/write.
- Parents can read only approved linked students and their permitted attendance information.
- Teachers can access only assigned classes unless granted an institute-level permission.
- Institute Admins cannot read another institute's records.
- Only Super Admin can change system-level institute controls.
- Passwords, SMS API keys and Firebase Admin credentials must never be stored in Firestore or mobile code.
- Use environment configuration and secret management for backend credentials.
- Store only necessary parent contact information and mask it in logs/UI where full display is not required.
- Do not store student photographs.
- Archive records rather than destructive deletion where historical integrity matters.
- Create audit logs for account changes, corrections, SMS controls and institute suspension.

11. Notifications and Backend

```
Teacher scan
  -> Firestore transaction saves attendance
  -> trusted backend detects/handles event
  -> backend resolves approved parent devices
  -> FCM sends push notification
  -> notification document/log is stored
  -> optional SMS check runs separately
```

- FCM is the default alert method.
- A trusted backend must send push notifications; do not embed server credentials in either app.
- Notification sending should be idempotent to prevent duplicate alerts.
- Retry transient failures and record permanent failures.
- SMS failure must never roll back attendance.
- Create deep links/navigation payloads that identify student, class and attendance record without exposing private data in the visible notification.

12. Cost and Free-Usage Design

Component	Planned cost approach
Flutter, Dart, Android Studio, Git	Free
Firebase Authentication - email/password	Use free service limits; avoid phone OTP
Cloud Firestore	Use free quota initially; optimize reads/writes
Firebase Cloud Messaging	No per-message charge for push notifications
QR generation/scanning	Free Flutter packages
PDF/CSV generation	On-device, free packages
Student photos/storage	Excluded
SMS	Optional paid feature, controlled per institute
APK distribution	Free direct distribution
Google Play publication	Optional one-time developer registration fee
User internet/mobile data	Paid by users according to their own provider

Important accuracy note

Free quotas, cloud prices and third-party SMS prices can change. Before production launch, verify the current official pricing and quotas for the selected Firebase region and SMS provider. The software must expose usage controls and must not assume unlimited free capacity.

13. Repository Structure

```
hamdhytech_attendance_system/
|-- AGENTS.md
|-- README.md
|-- apps/
|   |-- main_app/
|   `-- parent_app/
|-- packages/
|   `-- attendance_core/
|       |-- lib/models/
|       |-- lib/services/
|       |-- lib/repositories/
|       |-- lib/validation/
|       `-- test/
|-- firebase/
|   |-- firestore.rules
|   |-- firestore.indexes.json
|   `-- functions/ or backend/
|-- docs/
|   |-- architecture.md
|   |-- database.md
|   `-- release-checklist.md
`-- scripts/
```

14. Recommended Flutter Engineering Standards

- Use null-safe Dart and current stable Flutter.
- Use a clear architecture such as feature-first clean layers: presentation, domain and data.

- Use a predictable state-management solution consistently.
- Use typed immutable models and explicit serialization.
- Wrap Firebase behind repositories/services to make testing possible.
- Use dependency injection rather than global mutable singletons.
- Centralize routes, themes, validation, error mapping and logging.
- Never log passwords, tokens, full phone numbers or secret values.
- Use localization-ready strings from the beginning.
- Use automated formatting, linting, static analysis and tests in CI.

15. Development Phases for Codex

Phase	Deliverables
1. Foundation	Monorepo, two Flutter apps, shared package, linting, tests, README and AGENTS.md.
2. Firebase setup	App registration placeholders, Auth, Firestore repositories, emulator support, security-rule skeleton and indexes.
3. Authentication/roles	Login, session restoration, forced password change, role routing and suspended-account handling.
4. Super Admin	Institute creation, institute list/details, Institute Admin provisioning and controls.
5. Institute Admin	Teacher, permission, class and student management.
6. QR generation	Secure token creation, QR view, regeneration and PDF card.
7. Scanner/attendance	Continuous scanner, validation, sessions, entry/departure, duplicate protection and correction.
8. Parent App	Registration, secure linking, dashboard, history and multiple children.
9. Push notifications	Token management, trusted backend, FCM, logs, deep links and retries.
10. Optional SMS	Provider abstraction, institute controls, templates, quota tracking and logs.
11. Reports	CSV/PDF exports, filters and summaries.
12. Hardening/release	Security tests, emulator tests, performance, accessibility, release builds and documentation.

16. Acceptance Tests

- Super Admin can create an institute and initial Institute Admin.
- Institute Admin cannot access another institute even by changing IDs manually.
- Teacher sees only assigned classes unless granted additional permission.
- Parent cannot access a student using only institute code and student number.
- Primary parent mobile number is required when creating a student.
- Student photo is not requested or stored anywhere.
- Each QR contains no readable private student or parent data.
- Teacher opens scanner once and records multiple students without pressing scan/confirm repeatedly.
- Latest scanned student details appear below the scanner while it remains ready.
- Duplicate entry is rejected.
- Departure before entry is rejected.
- Attendance is saved using authoritative time and correct institute/class/student IDs.
- Attendance remains saved when push or SMS delivery fails.
- Push notification opens the correct child attendance record.

- SMS is not sent when Super Admin disables it.
- SMS is not sent when student SMS consent/preference is disabled.
- Push remains available when SMS is disabled.
- Manual attendance corrections require a reason and create an audit log.
- Parent App supports multiple linked children.
- No service secrets are present in the APK or committed repository.
- flutter analyze and flutter test pass for all packages and apps.

17. Required Automated Tests

- Unit tests for student/mobile validation and QR token parsing.
- Unit tests for entry/departure state transitions.
- Unit tests for duplicate-scan cooldown and idempotency keys.
- Repository tests using Firebase Emulator Suite.
- Security Rules tests for every role and cross-institute denial.
- Widget tests for login, dashboard states, student form and scanner feedback.
- Integration tests for institute -> teacher -> class -> student -> QR -> attendance workflow.
- Integration tests for parent linking and notification deep linking.
- Backend tests confirming one attendance event produces at most one push/SMS event.

18. AGENTS.md Content for the Repository

```
# HamdhyTech Attendance System

## Product
Two Android-first Flutter apps connected to one Firebase project:
- Main App: Super Admin, Institute Admin and Teacher
- Parent App: Parent/Guardian

## Non-negotiable rules
- Read the project specification before changing code.
- Do not hardcode credentials or secrets.
- Do not store plain-text passwords.
- Do not implement student photo collection or storage.
- Primary parent mobile number is required for each student.
- Enforce institute isolation through Firestore Security Rules.
- Parent access requires an approved parent-student link.
- Continuous QR scanning must not require confirmation per student.
- Attendance saving must not depend on notification delivery.
- FCM push is default; SMS is optional and controlled by Super Admin.
- Keep Firestore operations efficient and add tests for security and attendance rules.

## Quality commands
flutter pub get
flutter analyze
flutter test
flutter build apk --debug

## Definition of done
```

UI, validation, security, loading/error states, tests and documentation are complete.

19. Initial Prompt to Give Codex

Copy this prompt with the PDF

Attach this PDF to Codex and use the following instruction as the first task.

You are building the HamdhyTech Student Attendance System.

Read the attached project specification completely and treat it as the primary requirements document. Create only Phase 1: the repository foundation.

Requirements for this task:
1. Create a Flutter monorepo with apps/main_app, apps/parent_app and packages/attendance_core.
2. Configure Material 3 themes and localization-ready strings.
3. Add strict linting, unit-test setup and a root README.
4. Create AGENTS.md from the specification.
5. Add placeholder environment configuration without committing secrets.
6. Add architecture and Firebase setup documentation.
7. Do not implement student photos, SMS integration or production credentials.
8. Run flutter analyze and flutter test, fix all errors, and report commands and results.
9. End with a file tree and the exact manual setup steps I must perform next.

Do not start later phases until Phase 1 is clean and tested.

20. Inputs the Owner Must Prepare

- Final app names and package identifiers.
- HamdhyTech logo and preferred brand colors, or permission for Codex to use a temporary text logo/theme.
- Firebase project and two registered Android app configurations.
- A secure Super Admin Firebase Authentication account created outside source code.
- Institute and class sample data for testing.
- Final SMS provider account only when the optional SMS phase begins.
- Android test devices and parent/teacher test accounts.
- A Git repository and branch policy.
- Privacy policy and institute consent wording before real student data is used.

21. Out of Scope for Version One

- Student photographs and cloud image storage.
- Face recognition, fingerprint attendance or biometric identification.
- Live GPS tracking.
- Online fee/payment collection.
- Phone OTP login.
- WhatsApp Business messaging automation.
- Direct Google Sheets synchronization.
- iOS production release unless specifically planned later.

- AI analytics or predictive attendance features.
- Complex multi-branch hierarchy beyond one institute with many classes.

22. Final Completion Checklist

Area	Completion requirement
Architecture	Two apps, shared core, one Firebase project and documented backend.
Security	Role/institute Rules tested; no secrets or plain-text passwords.
Main App	All three roles and continuous QR attendance complete.
Parent App	Secure linking, simple dashboard, history and push alerts complete.
Data	No photos; required parent mobile; efficient indexed queries.
Notifications	FCM reliable and logged; SMS optional and independently controlled.
Quality	Analysis, unit/widget/integration/security tests pass.
Release	Signed APKs, privacy policy, setup guide and recovery procedures prepared.